

Live market ingest worker (CLI, Redis, no HTTP API)

TITLE	
Time	4–6 hours take-home, or 90 minutes live (slim version at the end)
Deliverable	Git repo (or zip) + a short architecture note (1-2 pages)
How results are returned	<code>print</code> JSON to stdout . stderr is logs. No HTTP server in Lua.

This assignment matches how production Lua is used in a high-load host service: a **one-shot CLI process**, arguments in, JSON out, Redis for shared state, spawned many times by other services.

Context

You are not building a website.

A host (Python, systemd, Kubernetes Job, CI, another team's script) will start your Lua program **many times per second**. Each run is a **command**. The host reads **stdout**, checks the **exit code**, and may kill the process on timeout.

Your job is to:

1. Pull **live** digital-asset (crypto) market data from a public source.
2. **Normalize** it into a stable internal schema.
3. Use **process memory** for hot data where it makes sense.
4. Use **Redis** so many concurrent processes share rate limits, last-good snapshots, and locks.
5. Stay runnable under **heavy parallel invocation** from external services.

Forbidden: exposing results via a Lua HTTP API, OpenResty location, Flask, or similar. If the host cannot `lua market.lua fetch BTC` and get JSON on stdout, the submission fails.

Commands (CLI contract)

Implement a single entrypoint, for example:

```
lua market.lua <command> [args...]
```

Required commands:

```
lua market.lua fetch BTC,ETH,SOL
lua market.lua snapshot --symbols BTC,ETH
lua market.lua convert --from BTC --to USDT --amount 1.5
lua market.lua health
```

Optional (strong submissions):

```
lua market.lua daemon
```

`daemon` may keep the process alive and read **newline-delimited JSON commands on stdin** or a **Unix socket**. Still **no HTTP**.

Stdout vs stderr vs exit codes

- **stdout:** one JSON object (preferred), or line-delimited JSON if you document it. If you also print debug lines, the host must still be able to parse the **last JSON object** – document this.
- **stderr:** human logs (no secrets).
- **Exit codes:**
 - `0` – success (`ok: true`)
 - `1` – business / upstream error (`ok: false`, structured body still on stdout)
 - `2` – bad arguments
 - `3` – Redis required and unavailable (if you chose fail-closed)

Reject arguments that contain `;` or other shell metacharacters. Do not pass user input to a shell.

Environment

Read configuration from the environment (no silent magic hosts in production code):

- `REDIS_HOST`, `REDIS_PORT` (and password if you use one)
- `SOURCE_TIMEOUT_MS`
- optional `SOURCE_URL` for a mock/fixture in tests

Document `LUA_PATH` / how to run from another working directory.

Functional requirements

1. Live fetch and normalize

Pick **one** public market API (CoinGecko, Binance public REST, Kraken ticker, etc.). Document **why**.

Map vendor JSON to a **frozen** schema. Example success body:

```
{
  "ok": true,
  "schema": "ticker.v1",
  "as_of_unix": 1726900000,
  "source": "binance",
  "items": [
    {
      "symbol": "BTC",
      "quote": "USDT",
      "price": "67210.12",
      "volume_24h": "12345.67",
      "change_24h_pct": "-1.24"
    }
  ],
  "meta": {
    "cache": "miss",
    "redis_ms": 2,
    "http_ms": 180
  }
}
```

Rules:

- Treat money as **strings** (or integer + scale). Do not use naive floats for prices.
- One bad symbol or missing field must **not** crash the process. Skip the item and include a reason, or return a partial `items` list – document the choice.
- Vendor / network errors: stdout JSON like `{ "ok": false, "code": "SOURCE_UNAVAILABLE", "detail": "..."}` and a non-zero exit.
- Schema version (`ticker.v1`) so a second source can be added later without changing the print contract.

`snapshot` returns last-known normalized data from Redis (and/or memory), without necessarily hitting the vendor.

`convert` uses cached/normalized prices to convert an amount; if data is missing, fail with a stable `code`.

`health` prints JSON: Redis ping, last successful fetch time if known, process ok. Do not invent a fake always-OK health.

2. Memory

You must show you understand **process-local** vs **shared** state.

- Process-local: Lua tables, bounded LRU (or similar) of last N symbols, max entries / max bytes, eviction policy **written down**.
- Shared: Redis (see below).

If every invocation is a new process, local tables die with the process — that is fine, but **say so**. Then Redis is the real cache.

The optional **daemon** is the place to prove in-process memory across many commands without an HTTP server.

3. Redis (required)

Use Redis for coordination under many workers, not as a dump of the whole program.

Minimum:

Key pattern (example)	Purpose
<code>mkt:lock:source</code> or per-symbol lock	<code>SET NX</code> + TTL so not every process hits the vendor
<code>mkt:snapshot:{symbol}</code>	last good normalized ticker + TTL
<code>mkt:rl>window</code>	rate limit so 1000 workers do not DDoS the vendor
optional <code>mkt:inflight</code>	deduplicate in-flight fetches

You must handle Redis down: **fail closed** or **serve last-known / degraded**. Choose one and **justify it** in the architecture note.

Locks must expire (TTL). Explain what happens if the host kills Lua at 5 seconds while a fetch is in flight.

4. Load and coalescing

The host will look like this:

```
seq 1 200 | xargs -P 50 -I{} lua market.lua fetch BTC,ETH
```

A strong design **does not** perform 200 vendor HTTP calls for 200 identical fetches. Use locks + Redis snapshots so most processes **print cached JSON**.

Report in the README (or a short `LOAD.md`):

- wall-clock p50 / p95 for that command
- how many HTTP calls actually went to the vendor vs 200 invocations
- what you cap (timeouts, concurrent upstream calls)

Timeouts are required on HTTP and Redis. Unbounded retry loops fail the review.

5. Callable from external services

Document the spawn contract a Python/Go/bash host would use:

- argv
- env vars
- cwd
- stdout = payload, stderr = logs
- exit codes

A 10-line Python or bash wrapper that runs Lua and prints/parses the JSON is welcome. **Lua must not listen on a TCP HTTP port.**

Architecture note (required)

1-2 pages, with a simple diagram. Cover:

1. **Process model:** one-shot CLI vs optional daemon; who owns concurrency (the host).
2. **Data flow:** argv → validate → Redis → maybe HTTP → normalize → print.
3. **Failure taxonomy:** recoverable (retry Redis/HTTP), retryable with a cap, fatal (bad args, poison payload).
4. **Why results are stdout, not an HTTP API** (process lifecycle, timeouts, kill-on-cancel).
5. **Memory vs Redis:** what lives in the process vs what is shared.
6. **How you would add a second market source** without changing the print schema.

Weak: a single god-file and “I’ll put nginx in front.”

Strong: Lua is a **bounded worker**; the host owns pooling and cancellation.

Constraints

- Lua 5.3+ or LuaJIT — state which.
- Redis is required for the graded path.
- No secrets in the repo.

- README: how to start Redis, run one `fetch` , run `health` , run the load command.

90-minute live version

Drop `convert` and `daemon` . Keep:

- `fetch` + normalize + print JSON
- Redis `SET` with TTL + `SET NX` lock
- 429 / Redis-down behavior
- 20-line architecture
- A smaller load: 20 parallel fetches

How we grade

Area	Weak	Strong
Architecture	Unclear process model; HTTP in Lua	CLI worker, host-spawned, stable contracts, swappable source
Lua structure	Globals, one file	Modules: cli, source, normalize, redis, cache
Data	Float prices; silent crash	Versioned schema; string decimals; partial success
Redis	Unused or one blob key	Lock, TTL snapshot, rate limit; failure mode documented
Load	1 HTTP call per CLI	Coalescing; timeouts; bounded concurrency
Ops	Only works on your laptop	Env, exit codes, stdout vs stderr
Safety	Shell-concatenated args	Validation; no <code>;</code> injection

Automatic fail: HTTP as the only way to get results; ignoring Redis; crashing on one bad symbol.